CS3240

User-Centered Design

—

S2AY2025-26

Assoc Prof Bimlesh Wadhwa
bimlesh@nus.edu.sg

Materials used for this course are to be strictly used only for the purpose of this course. These are NOT to be distributed or forwarded to anyone or posted on social media without consent.

User-Centered Design

User-Centered Design (UCD) is **a design philosophy** and **process** in which the **needs, preferences, and limitations of end users are given extensive attention** at every stage of the design process.

- Focus on real users and their tasks
- Iterative design : continuous feedback and refinement
- Empirical measurement : observe and test with users
- Design for usability : effectiveness, efficiency, and satisfaction

User Centered Design (UCD) Process

An Iterative and Collaborative Process with cycles of Critical and Lateral Thinking

Research and Understand context of Use

Define Scope of work & Specify Requirements

Ideate and Prototype design solutions

Evaluate the solutions against Requirements

UCD Process Models

Design Thinking

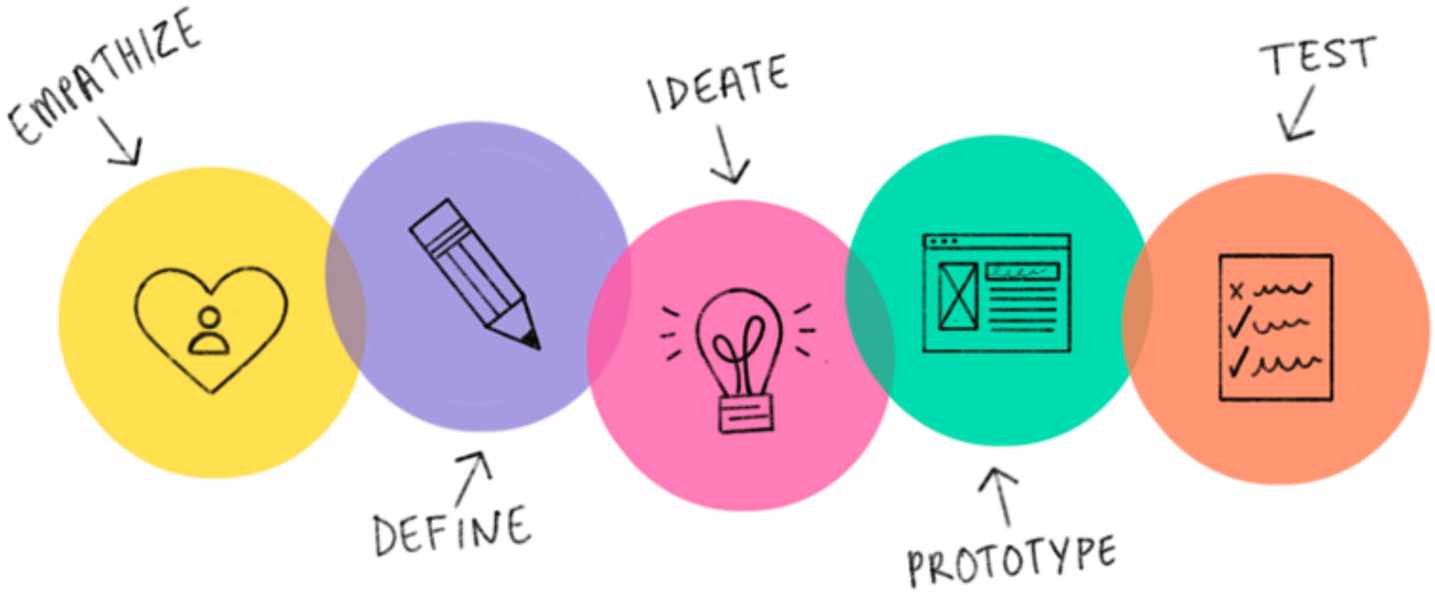
Contextual Design

Google Design Sprint

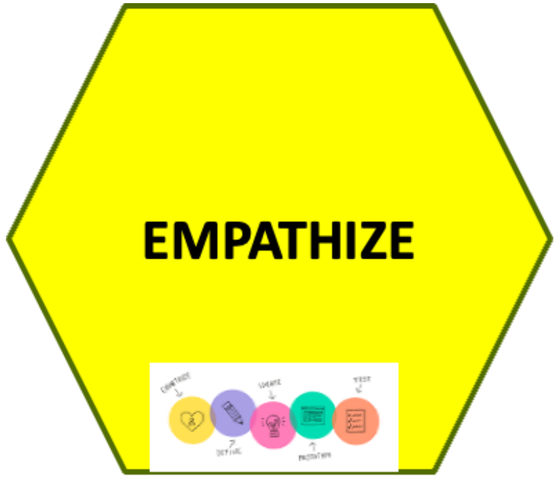
Double Diamond of Design

....

Design Thinking

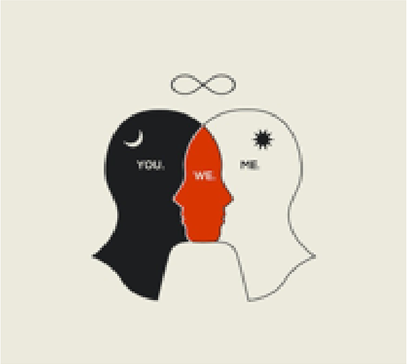


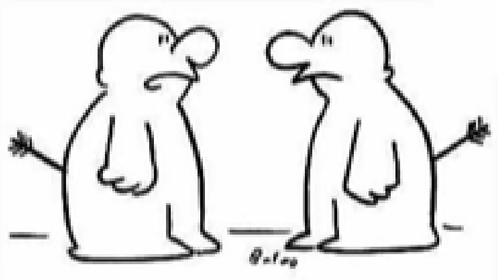
“Design thinking is a user-centered approach to innovation that draws from the designer's toolkit to integrate the needs of people, the possibilities of technology, and the requirements for business success.”
— Tim Brown, CEO of IDEO



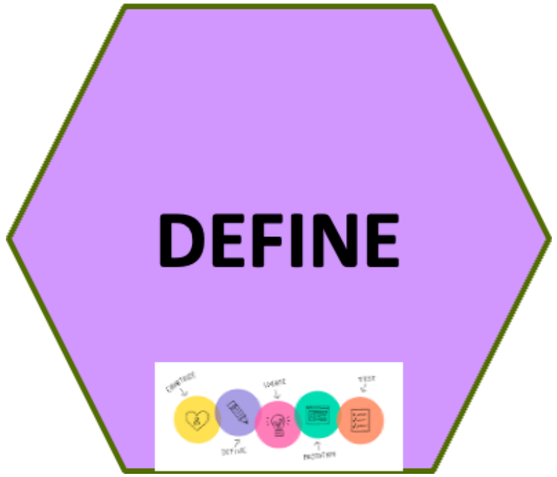
Research and Understand Context of Use

- Start with people.
- Dig into how work actually happens today — where effort piles up, what slows users down, and what success really looks like from their side.
- Map Tasks-To-Be-Done to surface the tasks your users need to accomplish
- Identify friction patterns: e.g. information overload, manual work, personalization gaps, adoption barriers, cross-system fragmentation.



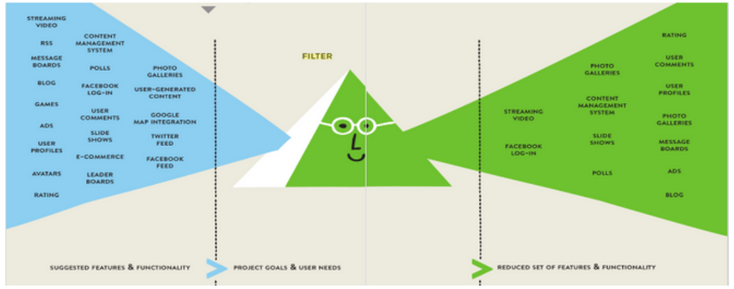


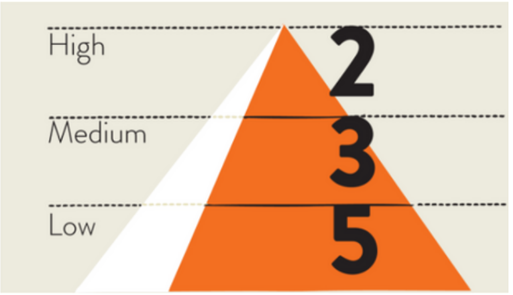
"I know exactly how you feel."

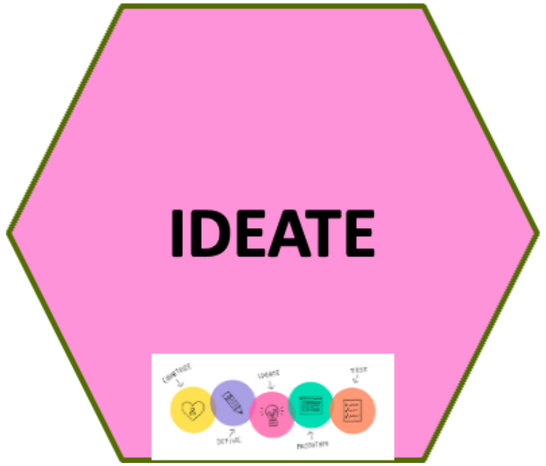


Define Scope of work Specify Requirements

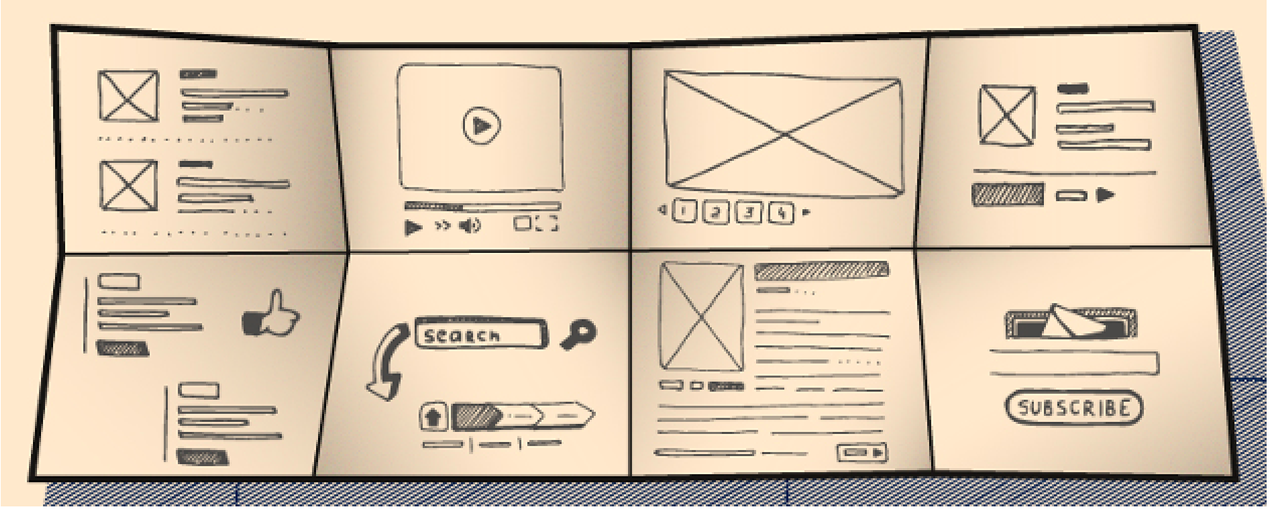
- Synthesize findings *(make it visual eg use Figjam or Miro boards)*
  - Team should see patterns emerging
- Identify opportunities :
  - What moves the needle on efficiency, effectiveness, satisfaction?
  - What can be built realistically with your data and current resources - including skills, time, tools?
  - Where does AI intervention feel natural vs. intrusive?
- Prioritize



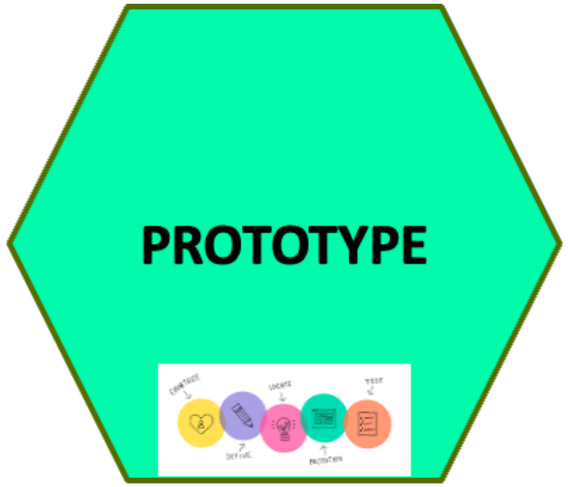




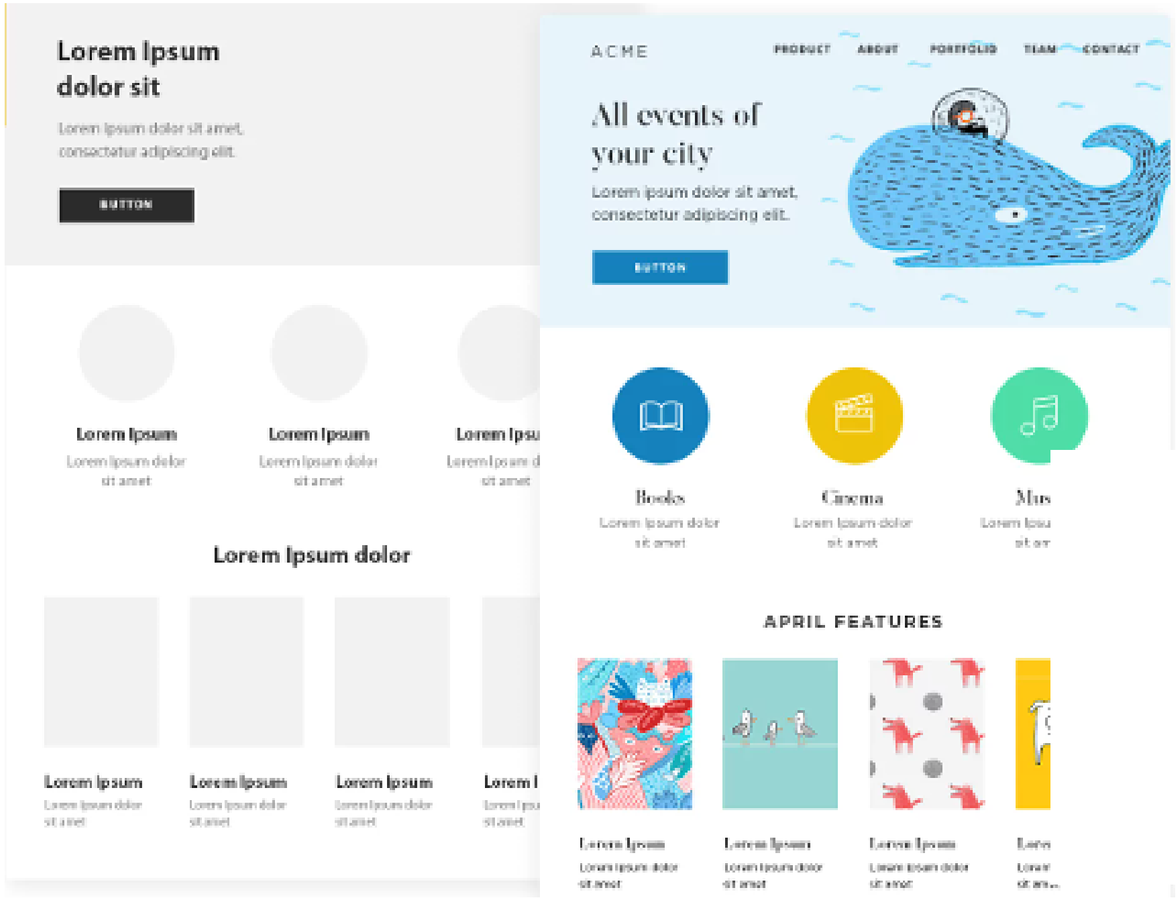
Brainstorm for solutions



- Once we understand and scope, we start design.
- This is where ideas take form.
- We define how the user should act, respond, and communicate
- We brainstorm, sketch, storyboard
  - interactions that mimic real user behavior,
  - then refine them through iterative feedback sessions.
  - Each round helps us understand not just what works, but why users would like it (or don't).



Build prototype(s)
Lo-fidelity -to- Hi-fidelity

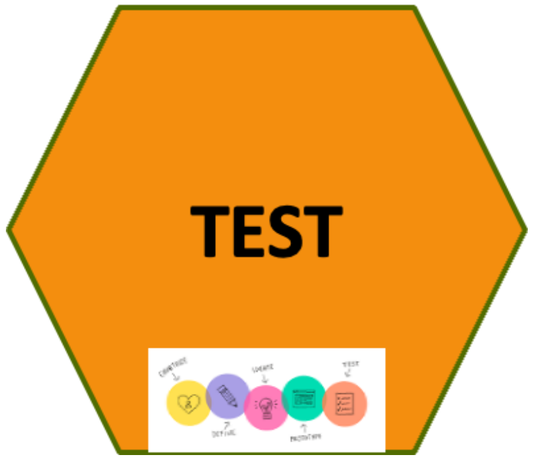


Wireframe

- LOW-FIDELITY
- BASIC LAYOUT
- STRUCTURAL GUIDELINES

Prototype

- ADVANCED
- VISUAL
- INTERACTION



Evaluate the prototypes against the Scope

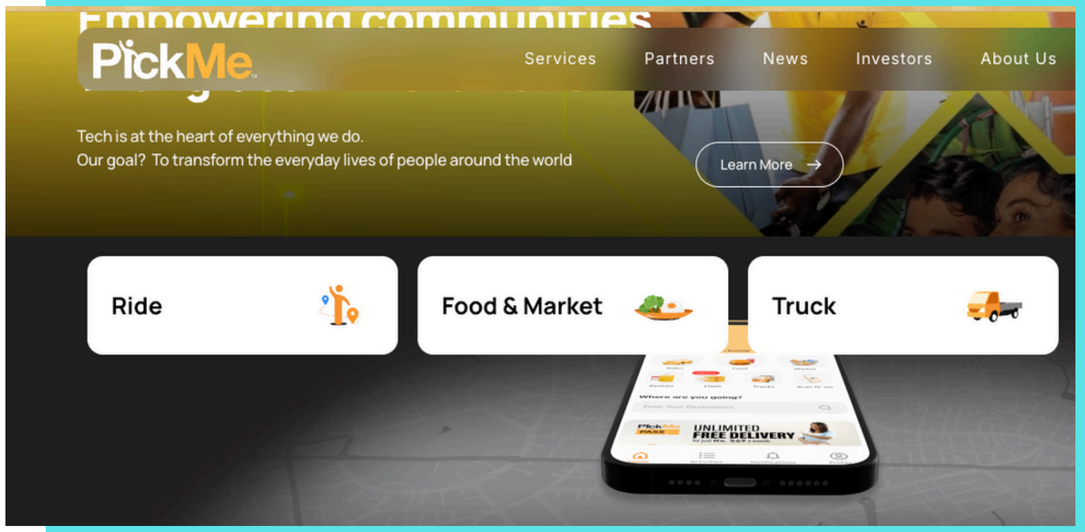
Real-World Example: PickMe - Sri Lanka

A great Sri Lankan example of design thinking is PickMe, the ride-hailing app.

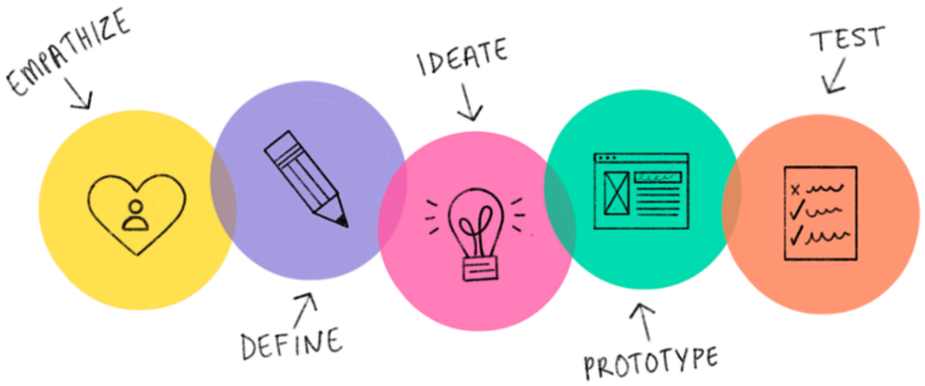
When PickMe launched, it didn't just copy foreign apps like Uber. Instead, they took time to **study the local market** and **talk to both drivers and passengers** in Sri Lanka. They learned:

- Many tuk-tuk drivers weren't used to using maps.
- Customers preferred to pay with cash, not credit cards.
- Internet coverage was weak in some areas.
- Users preferred apps in Sinhala or Tamil.
- People wanted options for bikes, cars, or three-wheelers based on budget and need.

Instead of building a fancy app first, PickMe listened to users, built simple prototypes, and tested features before launching. That's how they became one of Sri Lanka's most popular ride apps.



Does the UCD process change in anyway when the product we are designing a super-app?

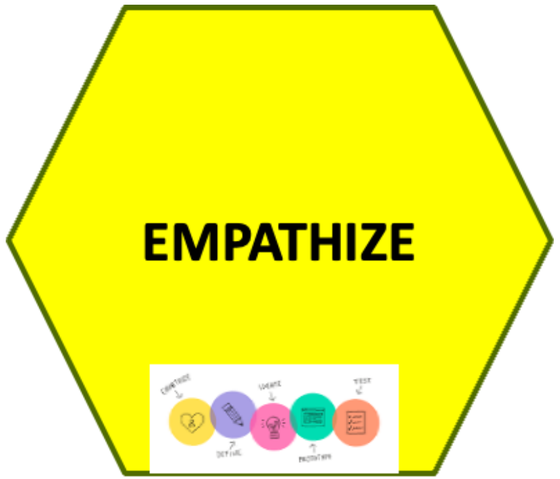


You still:
Understand users / Define user needs & problems / Design solutions /Evaluate with users /Iterate

The UCD process **does not change**, but
how you apply it does when designing a super-app.

What does change ?

A super-app introduces **scale, complexity, and diversity**



User research becomes **multi-layered/ complex**

Instead of “one user group, few main tasks,”
you may have

- Multiple user groups, multiple user goals (often unrelated), multiple services; different usage frequencies (daily vs occasional), cross-journey behavior (how users move between services) etc.

Grab super-app:

- *Daily commuters*
- *Food delivery users*
- *Small merchants*
- *Users with limited payment options*

DEFINE

Additional (not limited to):

- Scope shifts from tasks to **Services** and **Ecosystem**
- **Service discoverability**
- **Navigation** clarity and **perceived value** of “everything in one place”

Grab Super-app:

- *Scope*
  - *The problem is no longer booking a ride, but coordinating rides, food, and payments without repetition.*
    - *“Help users manage daily mobility, food, and payments through a single account and wallet.”*
    - *This leads to defining problems like:*
      - *Users want one payment system (GrabPay) that works across rides, food, and shopping*
      - *Users expect shared data (location, address, payment method) across services*
- *Service Discoverability :Users are unaware of or unable to discover relevant services beyond their primary ones*
  - *Which services should appear on the home screen?*
  - *Which services should be promoted vs hidden?*
  - *How to surface services contextually (e.g., food at meal times)?*
- *Navigation clarity : Users struggle to mentally map where services live within the app.*
  - *Clear service categories (Transport, Food, Finance)*
  - *Consistent entry points across services*
  - *Persistent access to GrabPay*
- *Perceived value: Users do not clearly perceive the value of using multiple services within Grab instead of separate apps.*
  - *Show savings through bundled promos*
  - *Highlight seamless transitions (food → wallet → ride)*
  - *Reinforce the idea of convenience and trust*

IDEATE

Additional ideation for (not limited to):

- **Navigation**
- **Design system**
- **Logical grouping** of services
- What appears first, what's hidden; **not showing everything at once**

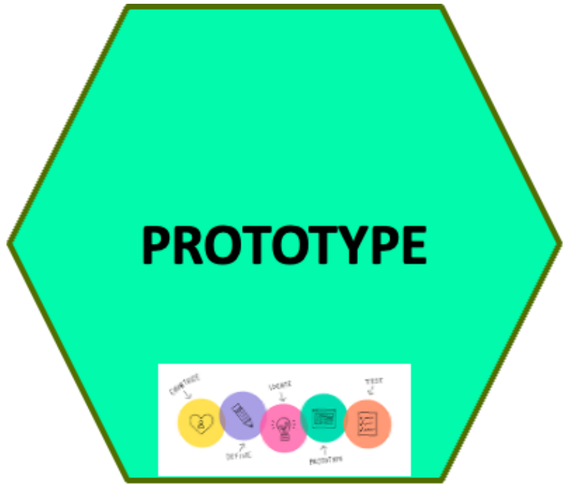
Grab Super-app *:*

- *Navigation*
  - *A single home hub instead of separate apps*
  - *Bottom navigation vs hub-and-spoke models*
  - *Persistent access to core utilities (wallet, notifications)*

- *Design System*
  - *Shared components across multiple services.*
    - *Universal button style for "Book", "Order", "Pay"*
    - *Shared interaction patterns for:*
      - *Address selection*
      - *Payment confirmation*
      - *Order tracking*

- *Grouping of Services*
  - *Grouping by user intent:*
    - *Transport (GrabRide, GrabBike)*
    - *Food & Delivery (GrabFood, GrabMart)*
    - *Finance (GrabPay, Insurance)*
  - *Alternatives*
    - *Grouping by frequency of use*
    - *Grouping by context (Now, Later)*

- *Progressive Disclosure*
  - *Showing most-used services first*
  - *Hiding less-used services under "More"*
  - *Contextual surfacing:*
    - *Food at lunch/dinner*
    - *Transport during commute hours*

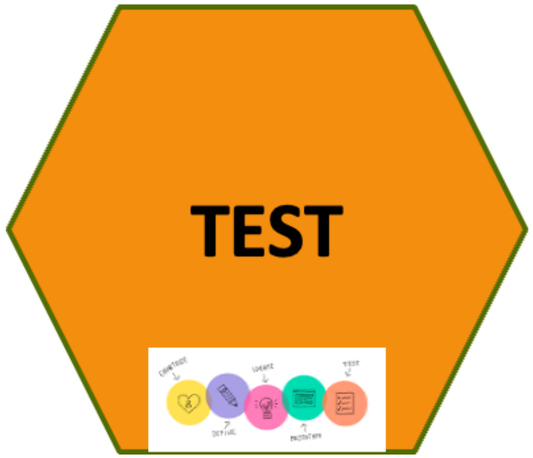


Instead of prototyping a few user tasks,
prototyping happens at **multiple levels**(not limited to):

- **Ecosystem-level** prototypes
- **Home screen** structure
- Global **navigation**
- **Entry points to services**
- **Service-level** prototypes
- **Moving from one service to another**
- **Shared actions** (login, wallet, notifications)

Grab Super-App

- *Ecosystem-level*
  - *Wireframe full app shell—home, navigation, wallet—before any single service flow is finalized.*
- *Home screen*
  - *Grid vs card-based services*
  - *Priority placement of GrabRide and GrabFood*
  - *Personalized sections like "Recommended for you"*
- *Global Navigation*
  - *Bottom navigation tabs (Home, Activity, Payments)*
  - *Persistent access to GrabPay*
  - *Consistent back and exit behavior across services*
- *Entry points to services*
  - *Different entry points to GrabFood:*
    - *Home screen icon*
    - *Promotional banners*
    - *Contextual prompts (meal times)*
- *Service-level prototypes*
  - *Booking a ride*
  - *Ordering food*
- *Moving from one service to another*
  - *Prototyped flows e.g. Order food → pay with GrabPay → book a ride*
  - *Ride completed → food recommendations appear*
- *Shared actions (login, wallet, notifications)*
  - *Single login for all services*
  - *GrabPay usage across rides and food*
  - *Unified notification for orders, rides, and promos*



Additional (not limited to):

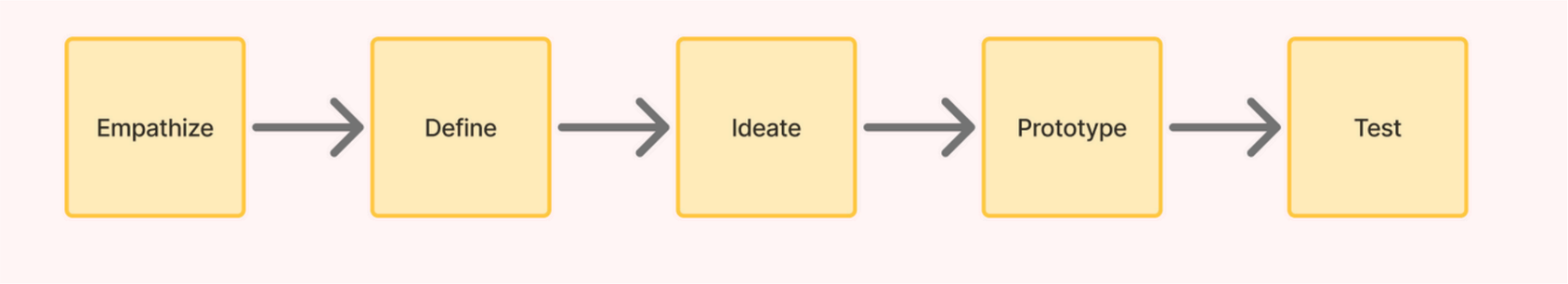
- Can users **find the right service**?
- Do **transitions between services** feel coherent?
- Do users understand they're still **in the same app**?

Grab Super-App

- *Can users find the right service?*
  - *"You want groceries delivered tonight—where would you go?"*
    - *Whether users locate GrabMart or GrabExpress?*
    - *Time taken to find the service*
    - *Whether users default to only GrabRide or GrabFood*
- *Do transitions between services feel coherent?*
  - *Scenario-based testing such as:*
    - *Order food → pay with GrabPay → receive a ride suggestion*
    - *Complete a ride → receive food or grocery recommendations*
      - *Whether transitions feel smooth or abrupt*
      - *Whether users feel disoriented when switching services*
      - *Whether UI patterns remain consistent*
- *Do users understand they're still in the same app?*
  - *During testing, designers observe:*
    - *Whether users say things like "I went to another app"*
    - *Whether users expect to re-login or re-enter payment details*
      - *Consistent branding, colors, and UI patterns*
      - *Unified wallet and notification behavior*
      - *Shared navigation elements across services*

Does the UCD process change in anyway when the product we are designing has an AI feature?

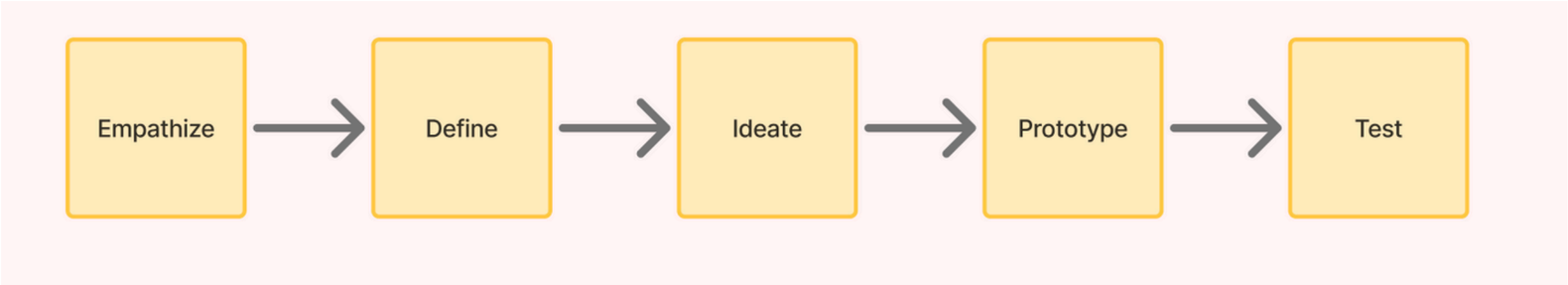
Traditional UCD process



With AI feature(s), Empathize - Ideate would need to factor in:

- What specific task(s) could AI help with?
- How AI fits into real task flows?
- Where AI can assist, automate, or amplify human judgment and where it shouldn't interfere.
- Which moments are repetitive, data-heavy, or delay-prone (ideal for AI)?
- Which require context, empathy, or expertise (best left human-led)?
- Define autonomy boundaries
  - High AI autonomy with light human oversight
  - AI suggests, human approves
  - Low AI autonomy, high human control
- Where AI adds clarity or speed, when it listens, recommends, summarizes, or acts , and how these moments connect to user intent.
- Define how the AI will learn from users, what signals it collects (edits, confirmations, rejections)

Traditional UCD process

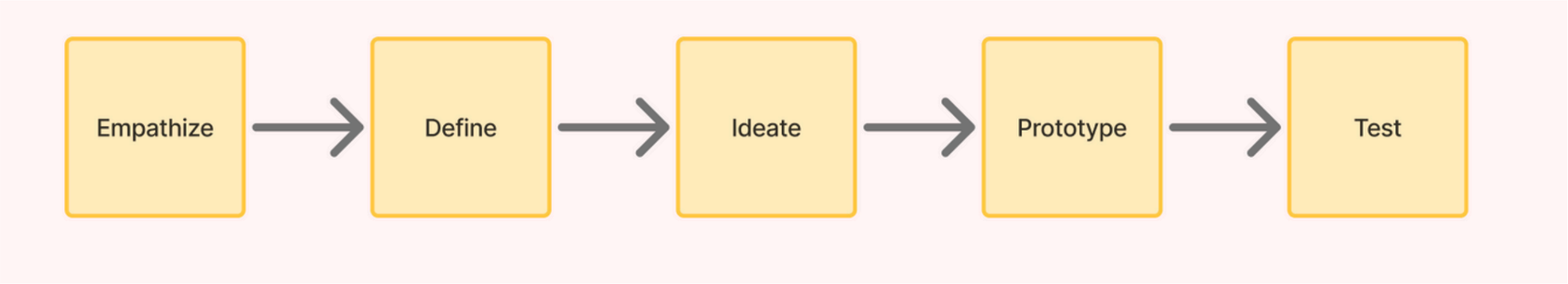


With AI feature(s), Prototype-Test would need to factor in:

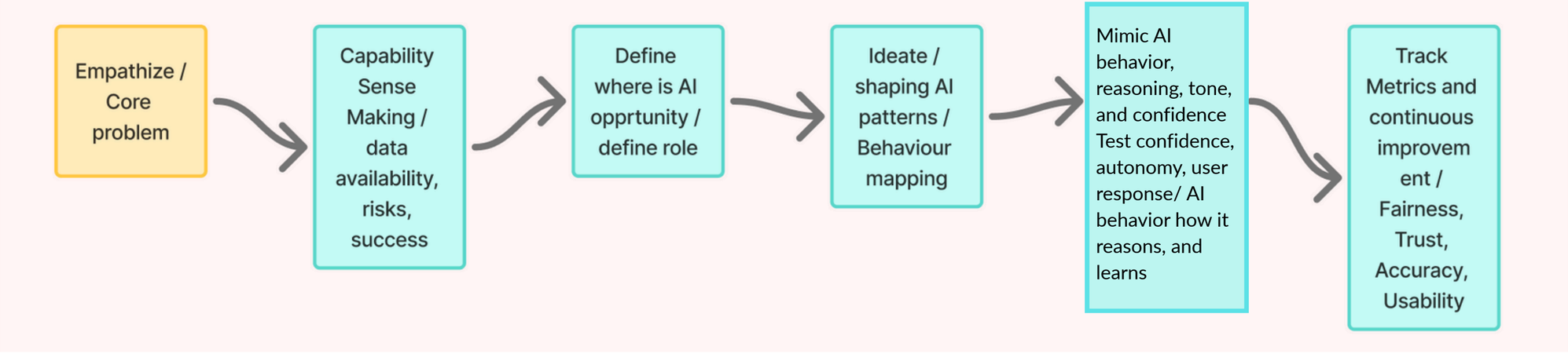
- Mimic AI behavior through simulated task flows
- Mimic AI reasoning, tone, and confidence
- Simulate AI to study real reactions without full engineering effort
- Test low-confidence or incorrect responses to understand recovery expectations
- Test if users grasp why AI suggested something and feel confident acting on it
- Autonomy testing between AI initiative and human oversight
- Test how users respond to AI suggestions i.e. what they correct, reject, or confirm
- Capture AI behaviours - how it reasons, hesitates, or learns from user

- In addition, Measure & Iterate
  - We observe how it performs, learns, and adapts - then tune the experience to stay accurate, dependable, and human-aligned.

Traditional UCD process



UCD process changes when designing for AI feature(s)



*Note: For CS3240 integrated design project you are **NOT** expected to follow above changed UCD process for introducing any AI feature in the super-app/service.*

Summary

- UCD process is a well established industry practice for interaction and experience design of products as well as services.
- There are many specific process models that follow the UCD Process framework.
- The process is infused with cycles of critical and lateral thinking, and is typically iterative, collaborative in nature with well defined outcomes at every stage.
- We shall follow a UCD process approach for CS3240.